X1 Vault: Encrypted IPFS Storage for Blockchain and AI Systems

Wallet-Derived Encryption, Content-Addressed Storage,

and Intelligent Deduplication for Autonomous Agents

X1 Labs
https://x1.xyz

February 2026 — Version 1.0

Abstract

We present X1 Vault, an encrypted storage system built on the InterPlanetary File System (IPFS) that provides end-to-end encrypted, content-addressed, and deduplicated file storage for blockchain ecosystems and artificial intelligence agents. X1 Vault derives encryption keys deterministically from wallet signatures, eliminating the need for password management while ensuring that only the key holder can access their data. We introduce a plaintext MD5 checksum protocol that enables intelligent deduplication across encrypted uploads, and describe how this architecture serves as persistent memory and archival storage for AI systems such as OpenClaw Bot. This paper details the cryptographic design, deduplication mechanism, API architecture, and practical applications across blockchain infrastructure and autonomous AI agents.

Contents

1 Introduction **3**

2 Background **3**
2.1 IPFS and Content Addressing . . . 3
2.2 The Encryption Challenge . . . 3
2.3 Wallet-Based Key Derivation . . . 4

3 System Architecture **4**
3.1 Architecture Overview . . . 4
3.2 Component Responsibilities . . . 4

4 Cryptographic Design **4**
4.1 Key Derivation . . . 4
4.2 Encryption . . . 5
4.3 Security Properties . . . 5

5 Deduplication Protocol **6**
5.1 The Problem . . . 6
5.2 Solution: Plaintext MD5 Protocol . . . 6
5.3 Security Considerations . . . 6
5.4 Deduplication Results . . . 6

6 Application: Blockchain Infrastructure **7**
6.1 On-Chain Data Archival . . . 7
6.2 Validator and Node Operator Logs . . . 7
6.3 Cross-Chain State Proofs . . . 7
6.4 Decentralized Identity Documents . . . 7

7 Application: AI Agent Memory and Archival **7**
7.1 The AI Memory Problem . . . 7
7.2 OpenClaw Bot: A Case Study . . . 8
7.3 Properties for AI Memory . . . 8
7.4 AI Archival Content . . . 8

8 API Design **9**

9 Backup Client **9**

10 Performance Characteristics **10**

11 Comparison with Alternatives **10**

12 Future Work **10**

13 Conclusion **10**

1 Introduction

The proliferation of blockchain networks and autonomous AI agents has created a growing need for decentralized, encrypted, and persistent storage systems. Traditional cloud storage solutions introduce central points of failure, require account management, and offer limited cryptographic guarantees about data confidentiality. Meanwhile, AI systems—particularly those operating as autonomous agents—require durable memory stores that persist across sessions, are verifiable, and remain under the control of their operators.

The InterPlanetary File System (IPFS) provides content-addressed storage where data is identified by its cryptographic hash rather than its location (1). This property makes IPFS inherently suitable for blockchain applications where immutability and verifiability are paramount. However, IPFS alone does not provide encryption, access control, or intelligent upload management.

X1 Vault addresses these gaps by combining:

- **Wallet-derived encryption**: AES-256-GCM keys derived deterministically from blockchain wallet signatures, requiring no passwords or key management infrastructure.
- **Content-addressed storage**: Files stored on IPFS with content identifiers (CIDs) that serve as tamper-evident references.
- **Plaintext MD5 deduplication**: A protocol for detecting duplicate content across encrypted uploads, eliminating redundant storage without compromising confidentiality.
- **An indexing layer**: A lightweight database mapping wallet public keys to their encrypted files, enabling file listing, search, and management.

2 Background

2.1 IPFS and Content Addressing

IPFS is a distributed, peer-to-peer file system that identifies content using cryptographic hashes called Content Identifiers (CIDs) (1). Unlike location-based addressing (HTTP URLs), content addressing ensures that the identifier of a piece of data is intrinsically tied to its content. Any modification to the data produces a different CID, providing built-in integrity verification.

Key properties of IPFS relevant to our design:

- **Immutability**: Once data is stored, its CID permanently references that exact content.
- **Deduplication**: Identical content stored by different users produces the same CID, reducing storage overhead at the network level.
- **Persistence through pinning**: Data remains available as long as at least one node pins it; unpinned data may be garbage collected.
- **Decentralized retrieval**: Content can be fetched from any peer that holds it, not just the original uploader.

2.2 The Encryption Challenge

While IPFS provides integrity and availability, it does not provide confidentiality. All data stored on IPFS is publicly readable by anyone who knows the CID. For sensitive data—private documents, AI agent memories, financial records—client-side encryption before upload is essential.

However, client-side encryption introduces a deduplication problem: encrypting identical plaintext with different random initialization vectors (IVs) produces different ciphertext, and therefore different CIDs. This means IPFS's native content-based deduplication no longer functions for encrypted data.

2.3 Wallet-Based Key Derivation

Blockchain wallets (Solana, Ethereum, X1) provide Ed25519 or secp256k1 key pairs used for transaction signing. We observe that these same key pairs can serve as the basis for encryption key derivation, eliminating the need for separate key management. The wallet *is* the encryption key—anyone with access to the wallet can derive the encryption key, and no one else can.

3 System Architecture

X1 Vault consists of four components: the client (browser or CLI), the vault server, the IPFS node, and the index database.

3.1 Architecture Overview

```
Client (Browser/CLI)          Vault Server          IPFS Node
=====================     ==================     ==============

1. Sign message    ----+
2. Derive AES key      |
3. Encrypt file        |
4. Compute MD5         |
5. Upload -------------|---> Proxy to IPFS -----> Store + Pin
                       |     Index in SQLite
                       |     Store MD5
                       |
6. List files ---------|---> Query index
7. Download + Decrypt -|---> Fetch from IPFS ---> Return data
```

3.2 Component Responsibilities

Client Handles all cryptographic operations: key derivation, encryption, decryption, and MD5 computation. The server never sees plaintext data.

Vault Server Proxies uploads to IPFS, maintains the file index (SQLite), and serves the web interface and API. It stores metadata—public keys, filenames, sizes, timestamps, and MD5 checksums—but never encryption keys or plaintext.

IPFS Node Stores and serves encrypted content, identified by CIDs. Provides pinning for persistence and peer-to-peer content distribution.

Index Database Maps wallet public keys to their files, enabling file listing, search, and deduplication queries without scanning the IPFS datastore.

4 Cryptographic Design

4.1 Key Derivation

The encryption key is derived in three steps:

1. The wallet signs a constant derivation message: `IPFS_ENCRYPTION_KEY_V1`.
2. The 64-byte Ed25519 signature is hashed with SHA-256 to produce a 32-byte key.
3. This key is used as the AES-256-GCM encryption key.

Formally, let sk be the wallet's secret key and $m =$ "IPFS_ENCRYPTION_KEY_V1":

$$K = \text{SHA-256}(\text{Ed25519-Sign}(sk, m)) \tag{1}$$

This derivation is **deterministic**: the same wallet always produces the same encryption key. This means a user can access their encrypted files from any device by connecting the same wallet, with no need to transfer or back up encryption keys separately.

4.2 Encryption

Files are encrypted using AES-256-GCM (Galois/Counter Mode), which provides both confidentiality and authentication. For each file:

1. Generate a random 12-byte initialization vector (IV).
2. Encrypt the plaintext with AES-256-GCM using key K and the IV.
3. Concatenate: IV‖ciphertext‖authTag (16 bytes).
4. Base64-encode the result.

The encrypted payload is wrapped in a JSON envelope:

```
{
  "version": 1,
  "algorithm": "AES-256-GCM",
  "wallet": "<public_key_base58>",
  "derivationMsg": "IPFS_ENCRYPTION_KEY_V1",
  "data": "<base64(IV + ciphertext + authTag)>"
}
```

4.3 Security Properties

- **Confidentiality**: AES-256-GCM with random IVs ensures that ciphertext is indistinguishable from random data.
- **Integrity**: The GCM authentication tag detects any tampering with the ciphertext.
- **No key transmission**: The encryption key never leaves the client. The server only sees ciphertext.
- **Forward secrecy caveat**: If the wallet's private key is compromised, all files encrypted with that wallet are decryptable. Users requiring forward secrecy should rotate wallets periodically.

5 Deduplication Protocol

5.1 The Problem

AES-256-GCM uses a random IV for each encryption operation. Encrypting identical plaintext twice produces different ciphertext:

$$E(K, IV_1, P) \neq E(K, IV_2, P) \quad \text{when } IV_1 \neq IV_2 \tag{2}$$

Therefore, IPFS assigns different CIDs to each upload, and the server cannot detect duplicates by comparing encrypted content. In our production deployment, this resulted in 285 files consuming 142 unique slots—approximately 50% of uploads were duplicates.

5.2 Solution: Plaintext MD5 Protocol

We introduce a client-side deduplication protocol using MD5 checksums of the plaintext *before* encryption:

1. Before encryption, the client computes $h = \text{MD5}(plaintext)$.
2. The client sends h in the `X-Content-MD5` HTTP header alongside the encrypted upload.
3. The server stores h in the index database alongside the CID and filename.
4. On subsequent backup runs, the client fetches all stored checksums via `GET /index/checksums?pubkey=.` and compares local file MD5s against the server index.
5. Files with matching filename and MD5 are skipped.

5.3 Security Considerations

MD5 is cryptographically broken for collision resistance (2). We use it here strictly as a **content fingerprint for deduplication**, not for security. An adversary who knows the MD5 of a plaintext learns almost nothing useful:

- The MD5 does not reveal the plaintext content (it is a one-way hash).
- The MD5 does not help decrypt the AES-256-GCM ciphertext.
- The MD5 reveals whether two files have identical content, which is acceptable for the user's own deduplication purposes.

SHA-256 could be substituted for MD5 if stronger collision resistance is desired, at the cost of longer checksums. For the deduplication use case, MD5's 128-bit output is sufficient.

5.4 Deduplication Results

In our production deployment with 285 indexed files:

Metric	Value
Total files in index	285
Unique filenames	142
Unique MD5 checksums	141
Duplicate uploads prevented per run	∼50%

6 Application: Blockchain Infrastructure

6.1 On-Chain Data Archival

Blockchain networks generate large volumes of data—transaction histories, smart contract states, governance proposals, validator logs—that must be preserved but are too large for on-chain storage. X1 Vault provides encrypted off-chain storage with on-chain references:

1. Encrypt the data with the operator's wallet.
2. Upload to IPFS via X1 Vault, receiving a CID.
3. Store the CID on-chain (32–46 bytes) as a permanent, verifiable reference.
4. Any party with the wallet can retrieve and decrypt the original data.

6.2 Validator and Node Operator Logs

Blockchain validators generate operational logs, performance metrics, and audit trails that are sensitive but must be preserved for compliance and debugging. X1 Vault's wallet-based encryption ensures that only the validator operator can access these logs, while the IPFS CID provides a tamper-evident reference that can be submitted to governance systems.

6.3 Cross-Chain State Proofs

In multi-chain ecosystems, proving the state of one chain to another often requires transmitting large Merkle proofs or state snapshots. These can be encrypted and stored on IPFS, with only the compact CID transmitted cross-chain. The receiving chain's bridge contract can verify the CID against the decrypted content.

6.4 Decentralized Identity Documents

Wallet-derived encryption naturally extends to decentralized identity (DID) systems. A user's identity documents, credentials, and attestations can be encrypted with their wallet and stored on IPFS. The user controls access by selectively sharing their public key or signing authorization tokens.

7 Application: AI Agent Memory and Archival

7.1 The AI Memory Problem

Large language model (LLM) based AI agents—such as OpenClaw Bot, autonomous coding assistants, and research agents—face a fundamental limitation: their context window is finite, and their knowledge resets between sessions. To maintain continuity, agents need persistent, secure, and structured memory stores.

Current approaches to AI memory include:

- **Local files**: Simple but not portable, not verifiable, and vulnerable to loss.
- **Cloud databases**: Centralized, require account management, and introduce trust dependencies.
- **Vector databases**: Optimized for semantic search but lack encryption and verifiability.

X1 Vault offers a fourth approach: **encrypted, content-addressed, deduplicated memory on IPFS**.

7.2 OpenClaw Bot: A Case Study

OpenClaw Bot is an AI agent that operates autonomously, performing research, code generation, and system administration tasks. Its memory requirements include:

Session logs Complete interaction histories for debugging and audit.

Learned patterns Insights about codebases, user preferences, and recurring problems.

Task state In-progress work, pending items, and decision rationale.

Knowledge base Domain-specific information accumulated over time.

Using X1 Vault, OpenClaw Bot stores its memory as encrypted files on IPFS:

```
# Agent backup workflow (runs after each session)
node vault-backup.js --dir ~/.openclaw/memory/

Output:
[1/12] SKIP  session-2026-02-13.md (unchanged)
[2/12] UP    patterns.md -> QmXxx... (md5:a1b2c3)
[3/12] NEW   task-queue.json -> QmYyy... (md5:d4e5f6)
...
Done: 3 uploaded, 9 skipped, 0 failed
```

7.3 Properties for AI Memory

X1 Vault provides several properties that are particularly valuable for AI memory systems:

Deterministic access The same wallet always derives the same encryption key. An agent can be restarted on any machine and immediately access its full memory by connecting its wallet.

Incremental backup The MD5 deduplication protocol ensures that only changed memories are uploaded. For an agent that modifies 2–3 files per session out of hundreds, this reduces upload volume by 95%+.

Tamper evidence Each memory file's CID is a cryptographic hash of its (encrypted) content. If any bit is altered, the CID changes. This allows agents to verify memory integrity.

Version history Each upload creates a new CID. The index database preserves all versions, allowing an agent to roll back to previous memory states.

Multi-agent isolation Different agents use different wallets. Agent A cannot read Agent B's memories, even though both are stored on the same IPFS node.

Portability Memories are not tied to any specific server or cloud provider. Any IPFS node can host the encrypted data, and any client with the wallet can decrypt it.

7.4 AI Archival Content

Beyond active memory, AI systems generate valuable archival content:

- **Training data snapshots**: Datasets used for fine-tuning, preserved with provenance via CIDs.
- **Model evaluation logs**: Performance metrics and test results across versions.

- **Research artifacts**: Papers, code, and experimental results that should be preserved immutably.
- **Conversation archives**: Historical interactions that may be referenced months or years later.

The combination of encryption (confidentiality), IPFS (permanence), and MD5 deduplication (efficiency) makes X1 Vault suitable for long-term AI archival where data volumes grow continuously but change rates are low.

8 API Design

The X1 Vault server exposes a RESTful API for file management:

Endpoint	Method	Description
`/api/v0/add?pin=true`	POST	Upload encrypted file (multipart). Headers: `X-Pubkey`, `X-Filename`, `X-Content-MD5`
`/index/files?pubkey=X`	GET	List all files for a wallet
`/index/file/{cid}`	GET	Get file metadata
`/index/file/{cid}`	DELETE	Remove from index and unpin from IPFS
`/index/checksums?pubkey=X`	GET	List plaintext MD5 checksums
`/index/register`	POST	Register a file manually
`/index/md5`	POST	Update MD5 for an existing CID

The upload endpoint proxies to the local IPFS node's `/api/v0/add` endpoint while simultaneously indexing the file metadata in SQLite. The `X-Content-MD5` header carries the plaintext MD5 for deduplication; if omitted, the server falls back to hashing the encrypted content (which is less useful for dedup but still provides integrity checking).

9 Backup Client

X1 Vault includes `vault-backup.js`, a command-line client that implements the full deduplication protocol:

```
# Backup specific files
node vault-backup.js notes.md config.json

# Backup a directory recursively
node vault-backup.js --dir ~/project/

# Dry run (check without uploading)
node vault-backup.js --dry-run --dir ~/documents/
```

The client operates in four phases:

1. **Key derivation**: Load wallet key, derive AES-256 key via signature + SHA-256.
2. **Remote index fetch**: `GET /index/checksums?pubkey=...` to build a local lookup table of filename → MD5 mappings.
3. **Comparison**: For each local file, compute MD5 and check against the remote index.
4. **Selective upload**: Only encrypt and upload files that are new or modified.

Output uses clear labels: `NEW` (first upload), `UP` (content changed), `SKIP` (unchanged), `FAIL` (error).

10 Performance Characteristics

Operation	Typical Time
Key derivation (Ed25519 sign + SHA-256)	< 1 ms
AES-256-GCM encryption (1 KB file)	< 1 ms
AES-256-GCM encryption (1 MB file)	~ 5 ms
MD5 computation (1 MB file)	~ 3 ms
IPFS add + pin (1 KB encrypted)	~ 50 ms
Checksums index fetch (285 files)	~ 3 ms
Full backup run (285 files, 95% cached)	~ 2 s

The dominant cost in a backup run is the checksums fetch and local MD5 computation, both of which are fast. Actual IPFS uploads only occur for new or modified files, making incremental backups highly efficient.

11 Comparison with Alternatives

Property	X1 Vault	S3+KMS	Filecoin	Arweave
End-to-end encrypted	Yes	Server-side	No	No
Wallet-derived keys	Yes	No	No	No
Content-addressed	Yes	No	Yes	Yes
Dedup (encrypted)	Yes (MD5)	No	No	No
No account needed	Yes	No	Yes	Yes
Free to use	Yes	No	No	No
AI agent friendly	Yes	Partial	No	Partial

12 Future Work

Semantic search over encrypted memory Integrate with vector embeddings computed client-side, enabling AI agents to perform semantic search over their encrypted memory stores without revealing content to the server.

Shared vaults Allow multiple wallets to access a shared encrypted namespace using key exchange protocols (e.g., ECDH), enabling team-based or multi-agent collaboration.

On-chain CID registry Publish CID roots to a blockchain smart contract, creating a verifiable, timestamped log of all backups.

Streaming encryption Support large file streaming encryption to handle multi-gigabyte datasets without loading entire files into memory.

IPNS for mutable references Use IPFS Name System (IPNS) to provide stable, human-readable references to the latest version of a file, simplifying AI agent memory retrieval.

13 Conclusion

X1 Vault demonstrates that encrypted, deduplicated, and decentralized storage can be achieved with minimal infrastructure by combining IPFS content addressing with wallet-derived encryption keys. The system requires no accounts, no password management, and no trust in the storage server—the wallet *is* the key.

For blockchain ecosystems, X1 Vault provides a natural complement to on-chain storage: large data lives encrypted on IPFS, referenced by compact CIDs stored on-chain. For AI

systems, it provides persistent, portable, and verifiable memory that survives across sessions, machines, and infrastructure changes.

The plaintext MD5 deduplication protocol solves a practical problem unique to encrypted storage systems, reducing redundant uploads by approximately 50% in production while preserving full confidentiality. Combined with the `vault-backup` CLI client, this enables efficient incremental backups for both human users and autonomous AI agents.

X1 Vault is open source and available at `https://vault.x1.xyz`.

References

[1] Benet, J. (2014). *IPFS — Content Addressed, Versioned, P2P File System.* arXiv preprint arXiv:1407.3561.

[2] Wang, X., & Yu, H. (2005). How to break MD5 and other hash functions. *Advances in Cryptology — EUROCRYPT 2005*, pp. 19–35.

[3] McGrew, D. A., & Viega, J. (2004). The security and performance of the Galois/Counter Mode (GCM) of operation. *Progress in Cryptology — INDOCRYPT 2004*, pp. 343–355.

[4] Bernstein, D. J., Duif, N., Lange, T., Schwabe, P., & Yang, B. Y. (2012). High-speed high-security signatures. *Journal of Cryptographic Engineering*, 2(2), 77–89.

[5] Yakovenko, A. (2020). *Solana: A new architecture for a high performance blockchain.* Solana Labs whitepaper.